Truffle

Your finances, unearthed — a voice-first personal finance assistant

System Architecture

pnpm + Turborepo monorepo · Next.js 14 · deployed on Vercel

System overview

A user speaks or types; Whisper transcribes; a LangGraph agent graph classifies intent and reasons over the user's real transaction history; a task-aware multi-provider LLM router picks the best available free-tier model; the answer streams back and is read aloud. Every LLM call is traced to Langfuse and metered in Postgres.

CLIENT — INSTALLABLE PWA (BROWSER)

Home · /
Dashboard, onboarding tour, CSV / receipt import, auth

Chat · /chat
Voice loop, streaming replies, proposal cards, proactive nudges

Insights · /insights
Goals, budgets, habits, heatmap, weekly & monthly reports

Next.js 14 App Router + RSC · Tailwind CSS · Framer Motion · **TanStack Query** server state · MediaRecorder → audio blob · **Web Speech API** — TTS playback · Supabase JS — auth session · next-pwa manifest / SW · PostHog browser SDK

Hooks: `useFinancialChat`, `useVoiceRecorder`, `useTextToSpeech`, `useTransactionsQuery`, `useTheme`

▼ HTTPS · bearer session token ▼

API — NEXT.JS ROUTE HANDLERS (VERCEL SERVERLESS, NODE RUNTIME)

`/api/voice`
multipart audio → Groq Whisper large-v3 → transcript

`/api/chat`
streamed chat + AI tool-calling, RAG context, Langfuse trace

`/api/parse-receipt`
image / PDF → Gemini vision → extracted transactions

`/api/transactions`
CRUD · anomaly detection · budget nudges

`/api/goals` · `/api/habits` · `/api/budgets`
CRUD · milestone / at-risk / streak nudges

`/api/insights`
snapshots · monthly AI report nudge

`/api/auth/demo`
mint one-time magic-link for demo account

`/api/chat/feedback` · `/api/account/delete`
score a reply · cascade data wipe

Every handler calls `requireUser()` — validates the Supabase JWT before any work. Proactive nudges fire inline (fire-and-forget) after writes; no cron.

▼ in-process import — `@truffle/ai` ▼

AI ORCHESTRATION — PACKAGES/AI

LangGraph agent graph
`graph.ts` — ragRetrieval → intentRouter → specialist agent → END

Proactive graph
`proactive.ts` — anomaly / habit / goal nudge branches

LLM Router
`router.ts` — task-aware provider selection, usage-gated fallback

Agents
intentRouter · spendingAnalyst · forecaster · affordabilityChecker · anomalyReviewer · savingsGoalAdvisor · habitAdvisor · synthesizer

Embeddings + vector store
`embeddings.ts` Gemini `gemini-embedding-2` (768d) · `vectorStore.ts` → `match_transactions` RPC

Eval & metering
`eval.ts` — `logEval()`, `incrementUsage()`, `getGlobalUsage()`

Tracing
`langfuse.ts` — OTel span processor (Langfuse SDK v5), tone detection, prompt builders

LLM PROVIDERS — FREE TIER, ROUTER-SELECTED

Groq · GPT-OSS 120B / 20B · Qwen 3.6 27B (vision) · Whisper large-v3

Gemini · 3.7 Flash · 3.5 Flash-Lite · embedding-2 · **Cerebras** · gpt-oss-120b · **OpenRouter** · Llama 3.3 70B (free)

Mistral · mistral-small-latest

DATA & PLATFORM

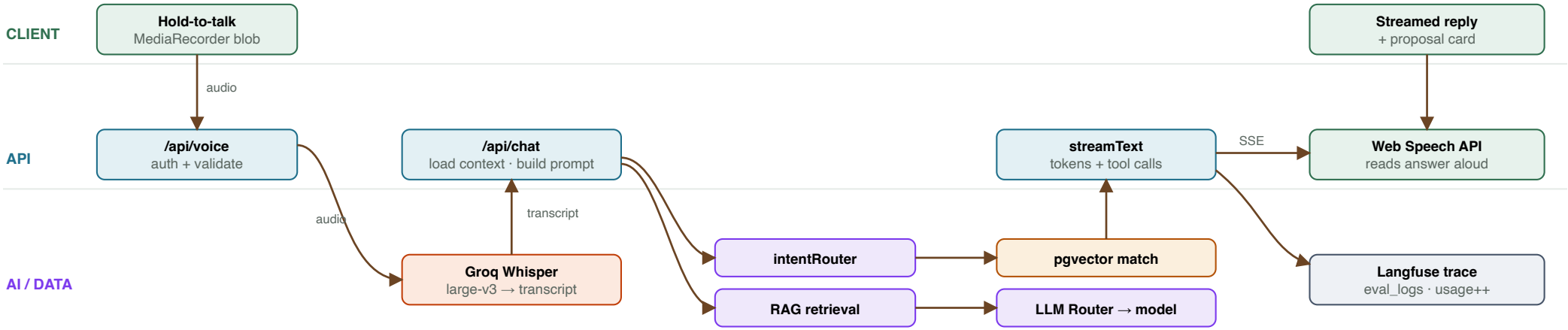
Supabase — Postgres + Auth + Storage · **pgvector** — HNSW cosine index

Langfuse — traces, generations, LLM-as-judge · **PostHog** — product analytics · **Vercel** — hosting / edge

GitHub Actions — CI + nightly demo reset

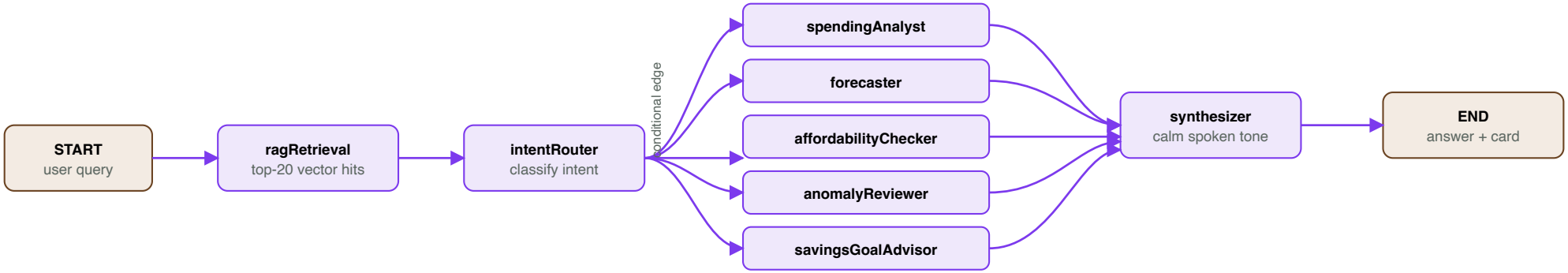
Client · API routes · AI orchestration · LLM providers · Data & platform

End-to-end voice request



The client sends recent context with every turn. `/api/chat` loads the last 50 transactions, snapshots, anomalies, goals, habits and budgets from Supabase in parallel, runs intent classification + semantic retrieval, builds a system prompt, then streams the reply. Tool calls (`proposeGoal`, `proposeTransaction`, `proposeHabit`) render confirmation cards the user taps to commit.

LangGraph agent graph — `packages/ai/src/graph.ts`



`routeAfterIntent()` maps the classified intent to exactly one specialist node. Each node calls `routedGenerateText(taskType, options)`, which selects a provider, runs the call, logs to `eval_logs`, and increments the daily usage counter. The `proactive.ts` graph reuses the same annotation and agents for unsolicited nudges (anomaly / habit-streak / goal milestone), written straight to `chat_messages` with a unique `nudge_key`.

Truffle

Router, data model, delivery

LLM Router · Persistence · Ops

Task-aware LLM router — `router.ts`

On every call the router reads today's global per-provider request counts from Postgres and walks the priority list for that task type, skipping any provider already at its daily budget. First one under budget wins; streaming callers get the full ordered candidate list and fall through on per-minute rate limits.

TASK TYPE	PRIORITY ORDER (FIRST AVAILABLE WINS)
`fast-chat`	Groq → Cerebras → OpenRouter → Mistral
`reasoning`	Gemini → Groq → Cerebras → OpenRouter
`tool-calling`	Groq → Gemini → Cerebras
`vision`	Gemini → Groq

PROVIDER	DAILY CAP	MODELS
Groq	900	`gpt-oss-120b / 20b · qwen3.6-27b · whisper-large-v3`
Gemini	1 300	`gemini-3.7-flash · 3.5-flash-lite · embedding-2`
Cerebras	1 500	`gpt-oss-120b`
OpenRouter	250	`llama-3.3-70b-instruct:free`
Mistral	600	`mistral-small-latest`

Counters are incremented via the race-safe `increment_llm_usage` Postgres RPC. Caps sit ~10% under real free-tier limits.

Observability & evaluation

- **Langfuse** (SDK v5, OTel) — a `chat` / `voice_transcription` trace per request, nested spans for `routeIntent` and `streamText` generations with token usage.
- **LLM-as-judge** — server-side `response-quality` evaluator scores production generations (`streamText`, `adviseHabit`, `adviseSavingsGoals`, `reviewAnomalies`) 1–5.
- **eval_logs** table — provider · task · input · output · latency · tokens per call.
- **PostHog** — pageviews and product events from the browser.
- **Golden benchmark** — `pnpm --filter @truffle/ai eval` runs 15 fixed queries; `eval:categorize` scores category-guidance accuracy (tunable with Weco).

Persistence — Supabase Postgres

TABLE	HOLDS
`transactions`	ledger rows + `embedding vector(768)`
`anomalies`	detected unusual charges (type, severity)
`monthly_snapshots`	per-month income / expense / category rollup (JSON)
`chat_messages`	conversation + proactive nudges (`is_proactive`, `nudge_key`, `read_at`)
`savings_goals`	target, saved, deadline, emoji
`savings_habits`	recurring weekly / monthly saving rule
`habit_contributions`	per-period contribution log (streaks)
`monthly_budgets`	per-category spending limit
`daily_llm_usage`	global per-provider request counter (per day)
`eval_logs`	every LLM call — latency, tokens, quality

RPC / EXTENSION	PURPOSE
`pgvector + HNSW`	cosine-distance ANN index on `transactions.embedding`
`match_transactions`	semantic retrieval for RAG context
`increment_llm_usage`	atomic, race-safe usage increment

Row-level security scopes every table to `auth.uid()`. Account deletion cascades via FK constraints. Auth = Supabase magic-link (passwordless); the demo account uses a server-minted one-time token.

Monorepo & delivery

PACKAGE	ROLE
`apps/web`	Next.js 14 PWA — pages, API routes, components, hooks
`apps/landing`	marketing site (Next.js, port 3001)
`packages/ai`	router, LangGraph, agents, prompts, embeddings, eval, Langfuse
`packages/db`	Supabase client + schema types + SQL migrations
`packages/types`	shared TS interfaces (Transaction, MonthlySnapshot, …)

- **Build** — pnpm workspaces + Turborepo; **deploy** — Vercel.
- **CI** (`ci.yml`) — type-check, lint, Vitest units, Playwright e2e on every push.
- **Nightly** (`demo-reset.yml`) — GitHub Action re-seeds the demo account from `transactions.csv`.